SIMATS ENGINEERING

ASSESSMENT TOOL 1 – High (Coding Challenge)

COURSE CODE: CSA0904 **COURSE NAME: Programming in Java**

COURSE OUTCOME COVERED

CO1: ***Apply object-oriented programming principles such as classes, objects, encapsulation, data hiding, inheritance, and polymorphism to design and implement entity manipulations (BL3,BL4)***

QUESTIONS: 5 **Total Marks: 100**

ANNEXURE A Coding Challenge

Code of Conduct:

I ***Kandath Haneesh(192324084)*** *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Clearly understands problem constraints, edge cases, and competitive requirements	Understands problem with minor gaps in constraints	Partial understanding; misses key constraints	Misinterprets problem or constraints
Algorithm	25%	Selects optimal algorithm with best time and space complexity for competitive constraints	Uses efficient algorithm with minor scope for improvement	Uses basic/inefficient approach	No clear algorithmic approach
Code Implementation	20%	Writes correct,	Code works with minor	Partially correct code; fails for	Code does not execute or

		efficient, and clean **code that passes all constraints**	**errors or** **inefficiencies**	**some cases**	**gives incorrect results**
Competitive **Performance**	**20%**	**Solves problem within time limits** **and handles large inputs effectively**	**Solves within acceptable time with minor delays**	**Struggles with time limits or larger inputs**	**Unable to solve within time constraints**
Test Case **Handling**	**15%**	**Handles all test cases including** **edge and hidden cases**	**Handles most test cases**	**Misses important edge cases**	**Fails on multiple test cases**

1. Library Book Management using Classes & Encapsulation

Design a Book class with private fields such as bookId, title, author, and price. Implement

constructors, getters, setters, and methods to issue and return books. Create a menu-driven program to manage multiple books using an array of objects. Also, construct suitable test cases to validate book addition, issue, return, and search operations.

Code:

```
import java.util.Scanner;
class Book {
    private int id;
    private String title;
    private String author;
    private double price;
    private boolean issued;

    Book(int id, String title, String author, double price) {
        this.id = id;
        this.title = title;
        this.author = author;
        this.price = price;
    }

    int getId() {
        return id;
    }

    void issueBook() {
        if (!issued) {
            issued = true;
            System.out.println("Book Issued");
        } else {
            System.out.println("Already Issued");
        }
```

```
    }

    void returnBook() {
        if (issued) {
            issued = false;
            System.out.println("Book Returned");
        } else {
            System.out.println("Book Not Issued");
        }
    }

    void display() {
        System.out.println("ID : " + id);
        System.out.println("Title : " + title);
        System.out.println("Author : " + author);
        System.out.println("Price : " + price);
        System.out.println("Status : " + (issued ? "Issued" : "Available"));
        System.out.println();
    }
}

public class LibraryManagement {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        Book[] b = new Book[10];
        int n = 0, ch;

        do {
            System.out.println("1.Add Book");
            System.out.println("2.Display Books");
```

```
System.out.println("3.Search Book");
System.out.println("4.Issue Book");
System.out.println("5.Return Book");
System.out.println("6.Exit");
System.out.print("Enter Choice: ");
ch = sc.nextInt();

switch (ch) {

   case 1:
      System.out.print("Book ID: ");
      int id = sc.nextInt();
      sc.nextLine();

      System.out.print("Title: ");
      String title = sc.nextLine();

      System.out.print("Author: ");
      String author = sc.nextLine();

      System.out.print("Price: ");
      double price = sc.nextDouble();

      b[n++] = new Book(id, title, author, price);
      break;

   case 2:
      for (int i = 0; i < n; i++)
         b[i].display();
      break;
```

```
case 3:
   System.out.print("Enter Book ID: ");
   int sid = sc.nextInt();

   for (int i = 0; i < n; i++)
      if (b[i].getId() == sid)
         b[i].display();
   break;

case 4:
   System.out.print("Enter Book ID: ");
   sid = sc.nextInt();

   for (int i = 0; i < n; i++)
      if (b[i].getId() == sid)
         b[i].issueBook();
   break;

case 5:
   System.out.print("Enter Book ID: ");
   sid = sc.nextInt();

   for (int i = 0; i < n; i++)
      if (b[i].getId() == sid)
         b[i].returnBook();
   break;

case 6:
   System.out.println("Thank You");
   break;
```

```
            default:
                System.out.println("Invalid Choice");
            }

        } while (ch != 6);

        sc.close();
    }
}
```

Output:

```
1.Add Book
2.Display Books
3.Search Book
4.Issue Book
5.Return Book
6.Exit
Enter Choice: 1
Book ID: 121
Title: rich dad poor dad
Author: jhon
Price: 320
1.Add Book
2.Display Books
3.Search Book
4.Issue Book
5.Return Book
6.Exit
Enter Choice: 2
ID : 121
Title : rich dad poor dad
Author : jhon
Price : 320.0
Status : Available
```

2. Employee Payroll System using Inheritance

Design a payroll system with a base class Employee and derived classes PermanentEmployee

and ContractEmployee. Override a method calculateSalary() in each subclass to compute salary differently. Display employee details and salary using runtime polymorphism. Also, construct suitable test cases to validate salary calculation for different employee types.

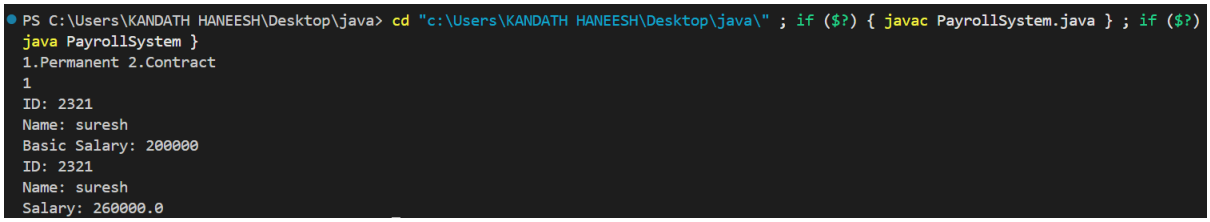
Code:

```
import java.util.Scanner;

class Employee {
    int id;
    String name;

    Employee(int id, String name) {
        this.id = id;
        this.name = name;
    }

    void calculateSalary() {}

    void display() {
        System.out.println("ID: " + id);
        System.out.println("Name: " + name);
    }
}

class PermanentEmployee extends Employee {
    double basic;

    PermanentEmployee(int id, String name, double basic) {
        super(id, name);
        this.basic = basic;
    }
```

```
    void calculateSalary() {
        display();
        System.out.println("Salary: " + (basic + basic * 0.3));
    }
}

class ContractEmployee extends Employee {
    int hours;
    double rate;

    ContractEmployee(int id, String name, int hours, double rate) {
        super(id, name);
        this.hours = hours;
        this.rate = rate;
    }

    void calculateSalary() {
        display();
        System.out.println("Salary: " + (hours * rate));
    }
}

public class PayrollSystem {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.println("1.Permanent 2.Contract");
        int ch = sc.nextInt();

        Employee e;
```

```
        System.out.print("ID: ");
        int id = sc.nextInt();
        sc.nextLine();

        System.out.print("Name: ");
        String name = sc.nextLine();

        if (ch == 1) {
            System.out.print("Basic Salary: ");
            double basic = sc.nextDouble();
            e = new PermanentEmployee(id, name, basic);
        } else {
            System.out.print("Hours Worked: ");
            int h = sc.nextInt();
            System.out.print("Rate Per Hour: ");
            double r = sc.nextDouble();
            e = new ContractEmployee(id, name, h, r);
        }

        e.calculateSalary();
        sc.close();
    }
}
```

Output:



3. Vehicle Rental System using Runtime Polymorphism

Develop a vehicle rental system with a superclass Vehicle and subclasses Car, Bike, and Bus. Override the method calculateRent(int days) in each subclass. Store vehicles in an array of Vehicle references and generate rental bills dynamically. Also, construct suitable test cases to validate rent calculation for different vehicle categories.

Code:

```
import java.util.Scanner;


class Vehicle {
    String name;

    Vehicle(String name) {
        this.name = name;
    }

    void calculateRent(int days) {
    }
}

class Car extends Vehicle {
    Car(String name) {
        super(name);
    }

    void calculateRent(int days) {
```

```
        System.out.println(name + " Rent = " + (days * 1000));
    }
}

class Bike extends Vehicle {
    Bike(String name) {
        super(name);
    }

    void calculateRent(int days) {
        System.out.println(name + " Rent = " + (days * 500));
    }
}

class Bus extends Vehicle {
    Bus(String name) {
        super(name);
    }

    void calculateRent(int days) {
        System.out.println(name + " Rent = " + (days * 2000));
    }
}

public class VehicleRental {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        Vehicle[] v = new Vehicle[3];

        System.out.print("Enter rental days: ");
```

```
        int days = sc.nextInt();

        v[0] = new Car("Car");
        v[1] = new Bike("Bike");
        v[2] = new Bus("Bus");

        System.out.println("\nRental Bills");
        for (Vehicle x : v) {
            x.calculateRent(days);
        }

        sc.close();
    }
}
```

Output:

```
PS C:\Users\KANDATH HANEESH\Desktop\java> cd "c:\Users\KANDATH HANEESH\Desktop\java\" ; if ($?) { javac VehicleRental.java } ; if ($?)
java VehicleRental }
Enter rental days: 6

Rental Bills
Car Rent = 6000
Bike Rent = 3000
Bus Rent = 12000
```

4. Bank Account System using Data Hiding

Design a BankAccount class with private fields accountNumber, holderName, and balance. Implement methods for deposit, withdrawal, and balance inquiry with proper validation. Prevent direct modification of balance from outside the class. Also, construct suitable test cases to validate valid and invalid transactions.

Code:

```
import java.util.Scanner;

class BankAccount {
    private int accountNumber;
```

```
private String holderName;
private double balance;

BankAccount(int accountNumber, String holderName, double balance) {
   this.accountNumber = accountNumber;
   this.holderName = holderName;
   this.balance = balance;
}

void deposit(double amount) {
   if (amount > 0) {
      balance += amount;
      System.out.println("Amount Deposited");
   } else {
      System.out.println("Invalid Amount");
   }
}

void withdraw(double amount) {
   if (amount > 0 && amount <= balance) {
      balance -= amount;
      System.out.println("Amount Withdrawn");
   } else {
      System.out.println("Insufficient Balance");
   }
}

void displayBalance() {
   System.out.println("Account Number: " + accountNumber);
   System.out.println("Holder Name: " + holderName);
   System.out.println("Balance: " + balance);
```

```
    }
}

public class BankSystem {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter Account Number: ");
        int acc = sc.nextInt();
        sc.nextLine();

        System.out.print("Enter Holder Name: ");
        String name = sc.nextLine();

        System.out.print("Enter Initial Balance: ");
        double bal = sc.nextDouble();

        BankAccount b = new BankAccount(acc, name, bal);

        System.out.print("Enter Deposit Amount: ");
        b.deposit(sc.nextDouble());

        System.out.print("Enter Withdraw Amount: ");
        b.withdraw(sc.nextDouble());

        b.displayBalance();

        sc.close();
    }
}
```

Output:

```
"C:\Program Files\Java\jdk-17\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2026.2\lib\idea_rt.jar=62642" -Dfile.encoding=UTF-8 -classpath "C:\Users\
Enter Account Number: 1211222
Enter Holder Name: haneesh
Enter Initial Balance: 2500000
Enter Deposit Amount: 50000
Amount Deposited
Enter Withdraw Amount: 200
Amount Withdrawn
Account Number: 1211222
Holder Name: haneesh
Balance: 2549800.0
```

5. University Admission System using Method Overloading

Create an Admission class that overloads the method calculateFee() for:

Undergraduate students,

Postgraduate students,

Scholarship students.

Display the fee structure based on user input and compare the overloaded method executions.

Also, construct suitable test cases to validate each overloaded version.

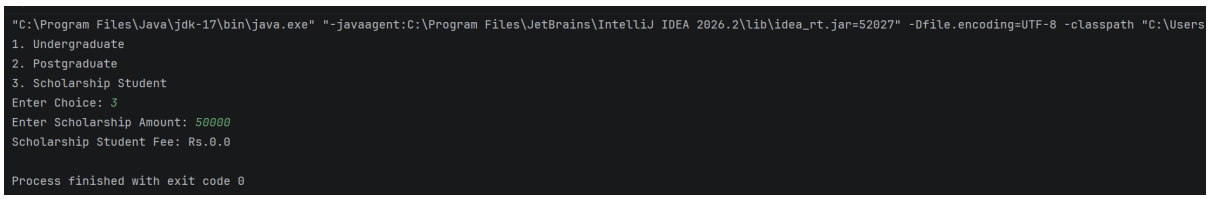
Code:

```
import java.util.Scanner;


class Admission {


    void calculateFee() {
        System.out.println("Undergraduate Fee: Rs.50000");
    }


    void calculateFee(int pg) {
        System.out.println("Postgraduate Fee: Rs.70000");
    }


    void calculateFee(double scholarship) {
        double fee = 50000 - scholarship;
        System.out.println("Scholarship Student Fee: Rs." + fee);
    }
}
```

```
public class UniversityAdmission {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        Admission a = new Admission();

        System.out.println("1. Undergraduate");
        System.out.println("2. Postgraduate");
        System.out.println("3. Scholarship Student");
        System.out.print("Enter Choice: ");
        int ch = sc.nextInt();

        switch (ch) {
            case 1:
                a.calculateFee();
                break;

            case 2:
                a.calculateFee(1);
                break;

            case 3:
                System.out.print("Enter Scholarship Amount: ");
                double amount = sc.nextDouble();
                a.calculateFee(amount);
                break;

            default:
                System.out.println("Invalid Choice");
        }
```

```
        sc.close();
    }
}
```

Output:



6. Shape Area Calculator using Abstract Class & Polymorphism

Design an abstract class Shape with an abstract method area(). Implement subclasses Circle, Rectangle, and Triangle. Use an array of Shape references to compute and display areas dynamically. Also, construct suitable test cases to validate area calculations and polymorphic behavior.

Code:

```
abstract class Shape {
    abstract void area();
}

class Circle extends Shape {
    double r;

    Circle(double r) {
        this.r = r;
    }

    void area() {
        System.out.println("Circle Area = " + (3.14 * r * r));
```

```
    }
}

class Rectangle extends Shape {
    double l, b;

    Rectangle(double l, double b) {
        this.l = l;
        this.b = b;
    }

    void area() {
        System.out.println("Rectangle Area = " + (l * b));
    }
}

class Triangle extends Shape {
    double base, height;

    Triangle(double base, double height) {
        this.base = base;
        this.height = height;
    }

    void area() {
        System.out.println("Triangle Area = " + (0.5 * base * height));
    }
}

public class ShapeArea {
    public static void main(String[] args) {
```

```
        Shape[] s = new Shape[3];

        s[0] = new Circle(5);
        s[1] = new Rectangle(4, 6);
        s[2] = new Triangle(8, 5);

        for (Shape x : s) {
            x.area();
        }
    }
}
```

Output:

```
"C:\Program Files\Java\jdk-17\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2026.2\lib\idea_rt.jar=50878" -Dfile.encoding=UTF-8 -classpath "C:\Users\
Circle Area = 314.0
Rectangle Area = 132.0
Triangle Area = 60.0

Process finished with exit code 0
```

7. Hospital Management System using Interfaces

Develop a hospital management system with an interface MedicalRecord containing methods addRecord() and displayRecord(). Implement the interface in classes Patient and Doctor. Demonstrate interface-based abstraction and object interaction. Also, construct suitable test cases to validate record creation and display.

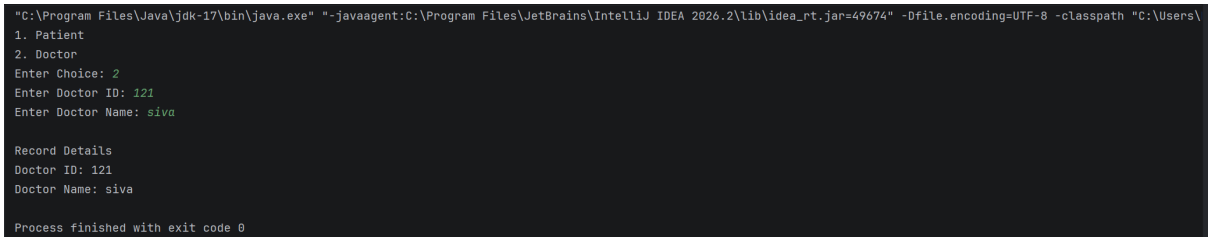
Code:

```
import java.util.Scanner;

interface MedicalRecord {
    void addRecord();
    void displayRecord();
}
```

```
class Patient implements MedicalRecord {
    int id;
    String name;

    public void addRecord() {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter Patient ID: ");
        id = sc.nextInt();
        sc.nextLine();
        System.out.print("Enter Patient Name: ");
        name = sc.nextLine();
    }

    public void displayRecord() {
        System.out.println("Patient ID: " + id);
        System.out.println("Patient Name: " + name);
    }
}

class Doctor implements MedicalRecord {
    int id;
    String name;

    public void addRecord() {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter Doctor ID: ");
        id = sc.nextInt();
        sc.nextLine();
        System.out.print("Enter Doctor Name: ");
        name = sc.nextLine();
    }
```

```
    public void displayRecord() {
        System.out.println("Doctor ID: " + id);
        System.out.println("Doctor Name: " + name);
    }
}

public class HospitalManagement {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.println("1. Patient");
        System.out.println("2. Doctor");
        System.out.print("Enter Choice: ");
        int ch = sc.nextInt();

        MedicalRecord m;

        if (ch == 1)
            m = new Patient();
        else
            m = new Doctor();

        m.addRecord();
        System.out.println("\nRecord Details");
        m.displayRecord();
    }
}
```

Output:



8. Online Shopping Order System using Packages

Create two packages:

shopping.product

shopping.order

Implement classes Product and Order in separate packages. Import the packages into a main application that places orders and calculates the total amount. Also, construct suitable test cases to validate package usage, order placement, and bill generation.

Code:

Order.java

```
package shopping.order;

import shopping.product.Product;

public class Order {

    Product p;
    int quantity;

    public Order(Product p, int quantity) {
        this.p = p;
        this.quantity = quantity;
    }

    public void bill() {
        System.out.println("Product : " + p.name);
        System.out.println("Price : " + p.price);
```

```
        System.out.println("Quantity : " + quantity);
        System.out.println("Total : " + (p.price * quantity));
    }
}
```

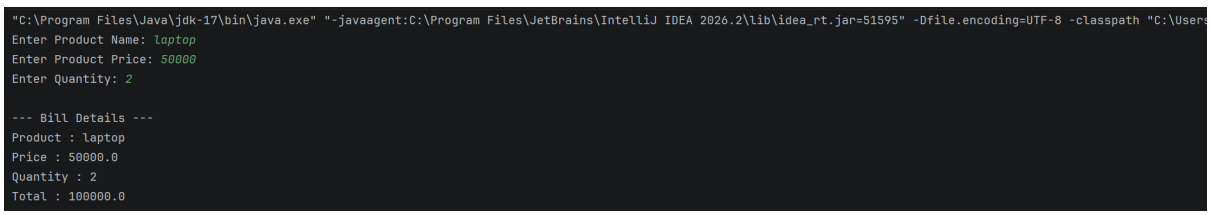
Product.java

```
package shopping.product;

public class Product {
    public String name;
    public double price;

    public Product(String name, double price) {
        this.name = name;
        this.price = price;
    }
}
```

Main.java

```
package shopping.main;

import shopping.product.Product;
import shopping.order.Order;

public class Main {
    public static void main(String[] args) {
        Product p = new Product("Laptop", 50000);
        Order o = new Order(p, 2);
        o.bill();
    }
}
```

Output:



9. Student Result Analyzer using Arrays of Objects

Design a Student class containing roll number, name, and marks in three subjects. Store multiple student objects in an array and implement methods to calculate total, average, grade, and class topper. Also, construct suitable test cases to validate grading and topper identification.

Code:

```
import java.util.Scanner;

class Student {
    int roll;
    String name;
    int m1, m2, m3;

    Student(int roll, String name, int m1, int m2, int m3) {
        this.roll = roll;
        this.name = name;
        this.m1 = m1;
        this.m2 = m2;
        this.m3 = m3;
    }

    int total() {
        return m1 + m2 + m3;
    }
```

```
    double average() {
        return total() / 3.0;
    }

    String grade() {
        double avg = average();

        if (avg >= 90)
            return "A";
        else if (avg >= 75)
            return "B";
        else if (avg >= 50)
            return "C";
        else
            return "Fail";
    }

    void display() {
        System.out.println("Roll No : " + roll);
        System.out.println("Name    : " + name);
        System.out.println("Total   : " + total());
        System.out.println("Average : " + average());
        System.out.println("Grade   : " + grade());
        System.out.println();
    }
}

public class StudentResult {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
```

```
System.out.print("Enter Number of Students: ");
int n = sc.nextInt();

Student[] s = new Student[n];

for (int i = 0; i < n; i++) {
   System.out.print("Roll No: ");
   int roll = sc.nextInt();
   sc.nextLine();

   System.out.print("Name: ");
   String name = sc.nextLine();

   System.out.print("Marks in 3 Subjects: ");
   int m1 = sc.nextInt();
   int m2 = sc.nextInt();
   int m3 = sc.nextInt();

   s[i] = new Student(roll, name, m1, m2, m3);
}

int max = s[0].total();
int topper = 0;

for (int i = 0; i < n; i++) {
   s[i].display();

   if (s[i].total() > max) {
      max = s[i].total();
      topper = i;
```

```
        }
    }

    System.out.println("Class Topper: " + s[topper].name);
    System.out.println("Topper Total: " + s[topper].total());

    sc.close();
  }
}
```

Output:

```
"C:\Program Files\Java\jdk-17\bin\java.exe" "-javaagent:C:\Program Files\JetBra
Enter Number of Students: 2
Roll No: 192324084
Name: haneesh
Marks in 3 Subjects: 99 99 99
Roll No: 192324100
Name: prasad
Marks in 3 Subjects: 90 92 99
Roll No : 192324084
Name    : haneesh
Total   : 297
Average : 99.0
Grade   : A

Roll No : 192324100
Name    : prasad
Total   : 281
Average : 93.66666666666667
Grade   : A

Class Topper: haneesh
Topper Total: 297

Process finished with exit code 0
```

10. Smart Home Device Controller using Hierarchical Inheritance

Develop a smart home system with a base class Device and derived classes Light, Fan, and AirConditioner. Implement methods to turn devices on/off and display power consumption. Use polymorphism to control devices through a common reference. Also, construct suitable test cases to validate device operations and power calculations.

Code:

```
class Device {
   String name;

   Device(String name) {
      this.name = name;
   }

   void on() {
      System.out.println(name + " ON");
   }

   void off() {
      System.out.println(name + " OFF");
   }

   void power() {
   }
}

class Light extends Device {
   Light(String name) {
      super(name);
   }
```

```
    void power() {
        System.out.println("Power Consumption: 20 W");
    }
}

class Fan extends Device {
    Fan(String name) {
        super(name);
    }

    void power() {
        System.out.println("Power Consumption: 75 W");
    }
}

class AirConditioner extends Device {
    AirConditioner(String name) {
        super(name);
    }

    void power() {
        System.out.println("Power Consumption: 1500 W");
    }
}

public class SmartHome {
    public static void main(String[] args) {

        Device[] d = new Device[3];
```

```
        d[0] = new Light("Light");
        d[1] = new Fan("Fan");
        d[2] = new AirConditioner("Air Conditioner");

        for (Device x : d) {
            x.on();
            x.power();
            x.off();
            System.out.println();
        }
    }
}
```

Output: